17/09/25

Practical - 6 Hamming Code

Aim: Write program to implement error detection and correction using hamming code concept. Make test to run input data stream & verify correction feature.

Code:

```
def calc_redundant_bits(m):
    for r in range(m+1):
        if (2**r) >= (m+r+1):
            return r

def main():
    data = input("Enter data bits: ").strip()
    m = len(data)
    r = calc_redundant_bits(m)
    n = m + r
    print("Number of r added: ", r)
    print("Total frame length: ", n)
    code = ['0'] * (n+1)
    j = 0
    for i in range(1, n+1):
        if (i & (i-1)) != 0:
            code[i] = data[-(j+1)]
            j += 1

    for i in range(r):
        parity_pos = 2**i
        val = 0
        for pos in range(1, n+1):
            if (pos & parity_pos) != 0
                and pos != parity_pos:
                if code[pos] == '1':
```

```
                    val ^= 1

        code[parity_pos] = str(val)

transmitted_str = "".join(code[1:][::-1])
print(f"Data to be transmitted: {transmitted_str}")
change = input("\nTransmitted bit change?").strip().lower()

if change == "yes":
    received_str = input(f"Enter corrupted data").strip()
    if len(received_str) != n:
        print(f"Error: Received data must be exactly {n} bits long!")
        return
    received_code = ['0'] + list(received_str[::-1])
else:
    received_code = list(code)
error_position = 0
for i in range(r):
    parity_pos = 2**i
    val = 0
    for pos in range(1, n+1):
        if (pos & parity_pos) != 0:
            if (received_code[pos] == '1'):
                val ^= 1

    if val != 0:
        error_position += parity_pos
if error_position == 0:
    print("\nVerification Result: No error detected.")
else:
    print("\nVerification Result: Error detected!")
    print(f"Wrong bit at position: {error_position}")
```

```
received_code[error_position] = '0'
if received_code[error_position] == '1' else '1'
corrected_str = "".join(received_code[1:][::-1])
print(f"corrected data stream : {corrected_str}")

if __name__ == "__main__":
    main()
```

Output :

① Enter data bits : 1011100
Number of redundant bits (r) needed : 4
total frame length (m+r) : 10
Data to be transmitted : 1011100011

Transmitted bit change ? (Yes/No) : Yes
Enter the corrupted Data Stream : 1011101011

Verification result : Error detected !
Wrong bit is at position : 4
Corrected data streams : 1011100011

② Enter data bits : 1011
Number of redundant bits (r) needed : 3
total data frame length (m+r) = 7
Data to be transmitted : 1010101

Transmitted bit change ? (Yes/No) : No
Verification Result : No error detected in transmission

Result :

Thus the Hamming code was implemented successfully for error detection and correction.